

基于模型的强化学习 · Model-based RL

世界模型的两条路线

以及我把其中一条拆开验证之后

Dreamer (生成式 RSSM) · LeWorldModel (非生成式 JEPA)

分歧在哪

Dreamer

“能画出来，才算真懂。”

世界模型带 decoder，训练目标包含把潜在状态还原成像素。然后让策略完全在这个模型的想象里训练。

代价：为了预测球往哪飞，顺带得学会画背景里每一片草叶。

JEPA / LeWorldModel

“预测像素本身就是错的目标。”

完全没有 decoder。把观测编码成向量，只预测下一个向量。规划放到测试时做——CEM 搜索动作序列。

代价：没有任何东西阻止 encoder 塌缩成一个常数。必须额外加约束。

两者不共享优化目标（奖励 vs 与目标的距离），也不共享评估指标（累计回报 vs 成功率）。这一点后面还会回来。

我做了什么

1

手写 Dreamer

从零实现，单文件。RSSM、reward model、actor-critic、imagination rollout。

2

复现 LeWorldModel

跑通官方实现，复现出 86% 的规划成功率。

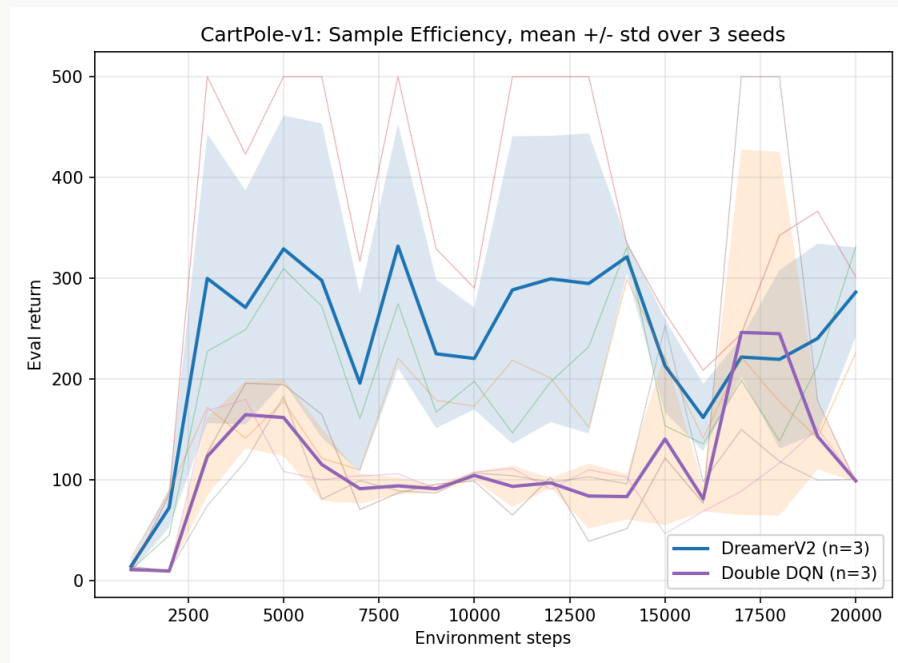
3

然后去验证它

四组受控实验。第四组推翻了前三组的结论——这才是值得讲的部分。

Dreamer，一行行写出来

约 670 行，不依赖任何框架。GRU 确定性状态 + 32×16 categorical 随机状态，straight-through 梯度，actor-critic 完全在想象轨迹上用 λ -return 训练。



240.1

Dreamer 平均回报

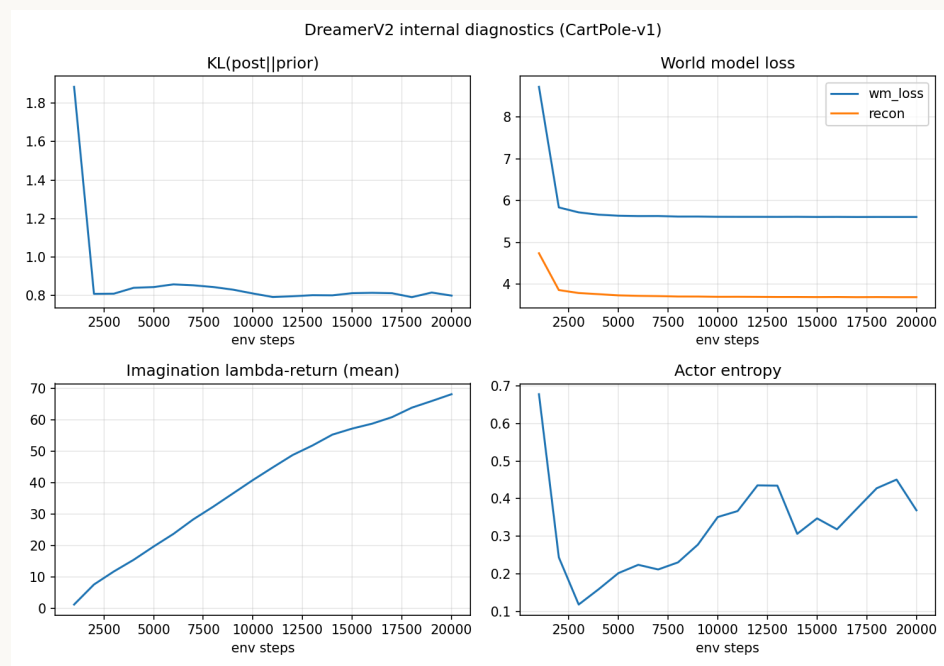
113.8

Double-DQN

CartPole-v1 · 2 万步 · 3 个 seed

想象出来的东西，对应现实吗？

actor 从头到尾没见过一个真实状态。所以关键问题是：想象轨迹到底跟不跟现实对齐， 还是说世界模型编了一个奖励很高但不存在的世界，策略在里面自我陶醉。



KL 稳定在 0.8 附近，free bits 正在起作用（既没后验塌缩也没爆炸）。想象中算出的 λ -return 从 0 涨到 70+，与真实评估回报同步——这就是"没有自欺"的证据。

LeWorldModel, 复现

86%

官方 checkpoint 的规划成功率

约 4 小时

花在装环境上，不是花在科研上

tworoom 任务，CEM-MPC，50 条留出 episode。这是后面所有数字的参照基准。

官方 README 和已发布的包之间有四处断裂：`box2d-py` 缺 `swig` 编译不过、`datasets` 解析到五年前的版本、`transformers` 5.x 重命名了所有 ViT 权重导致 checkpoint 加载失败、HDF5 支持被一个静默的 import 失败关掉了。这些我全写进了仓库——复现类文章通常略过这段，而这段恰恰是真正卡住人的地方。

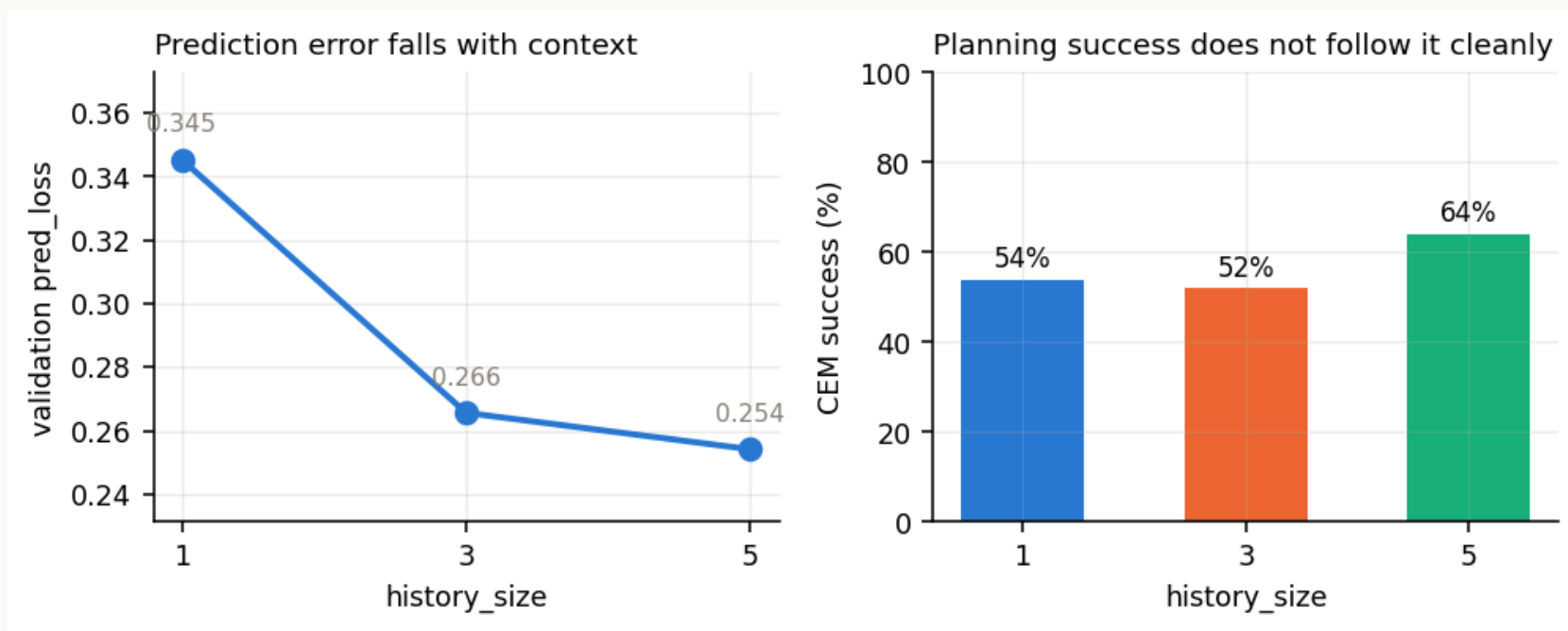
以上是复现

以下是我自己做的

同一份数据、同一个 encoder、同一个优化器、同样 900 步梯度更新、同样 50 条留出 episode。
每组实验只动一个变量——至少我当时是这么以为的。

实验一 —— 需要看多少历史？

`history_size` $\in \{1, 3, 5\}$, 即 predictor 能看到几帧过去的 embedding。3 是论文默认值。

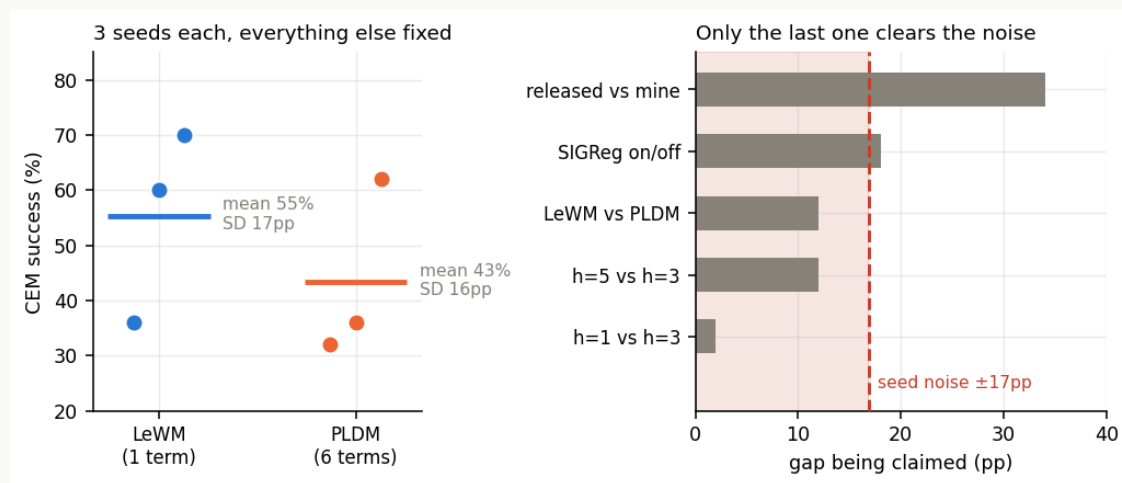


loss 随上下文单调下降，成功率却没跟上。 $h=1 \rightarrow h=3$ 这一步，loss 改善了，成功率反而掉了 2 个点。50 条 episode 的标准误差约 7 个百分点，所以这一对就是噪声，当时我留下的结论只有“ $h=5$ 更好”。记住这个成功率排序：**52 / 54 / 64** —— 第 14 页会回来推翻它。

实验二 —— 变量选错了，重做一次

原计划是"固定预算只换架构"。但这个设计本身就是错的：JEPA 是一个方法家族，成员共享架构，区别在 anti-collapse loss。我只换了模型类、loss 仍然是 LeWM 的 SIGReg——**两臂跑的是同一个方法**（两份训练日志都在报 `sigreg_loss`，真 PLDM 不该有这一项）。

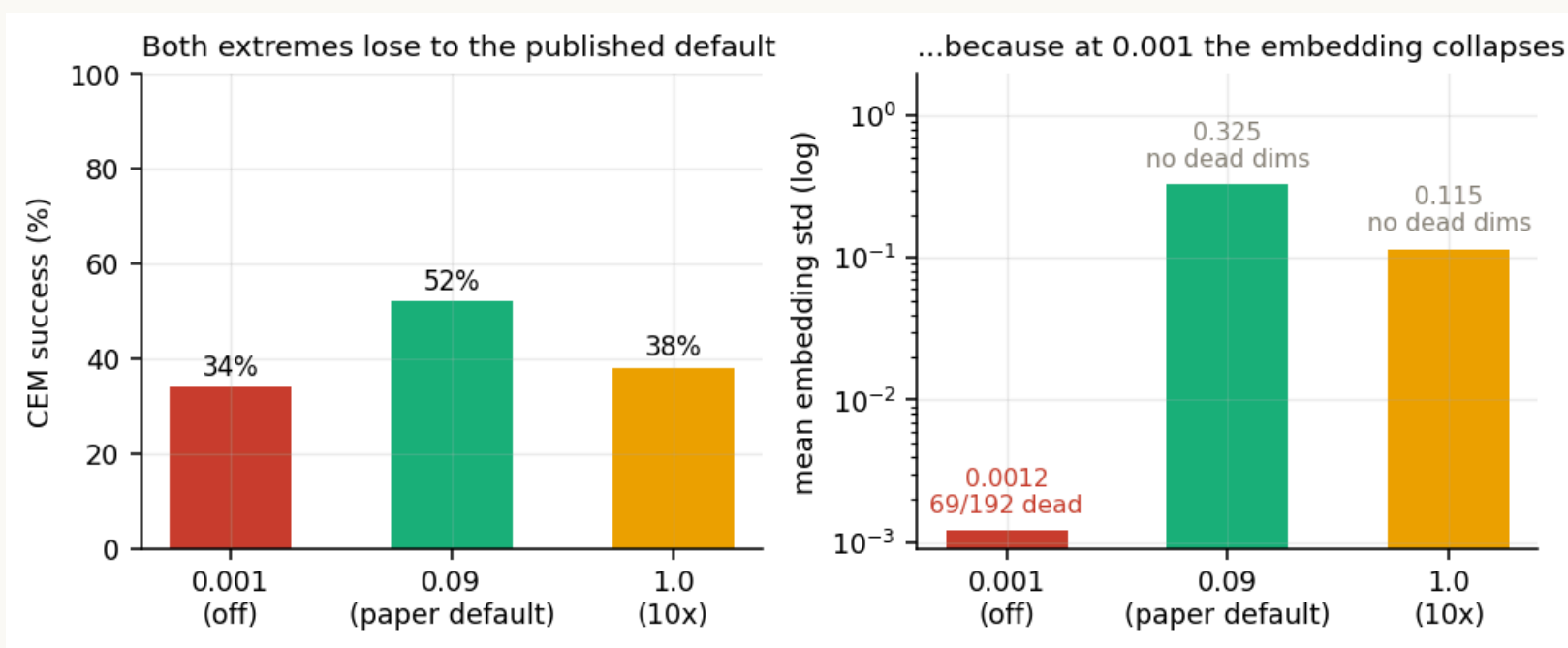
重做：架构、数据、预算、规划器全部固定，**只换防塌缩机制**（SIGReg×1 对 VReg×4 + 时序对齐 + 逆动力学 = 6 项），补上 `Manager` 的 seed，每臂 3 个 seed。



LeWM 55.3% 对 PLDM 43.3%， $t = 0.87$ ，CI [-27, +51]——方向和论文一致，但完全不显著。真正的产出是那个标准差：只换 seed，成功率能从 36% 走到 70%， ± 17 个百分点。要在 80% 功效下测出 10 个点的差异，每臂约需 45 个 seed。我有 1 个。

实验三 —— 于是我不再信 loss

LeWorldModel 的核心主张：两项 loss 就够，因为 SIGReg 会防止表征塌缩。我把它的权重扫到 0.001 和 1.0，并且没有去读 loss，而是直接测量 embedding 本身。



权重降到 0.001 时：**pred_loss = 0.004**，是我整晚训出来的最低值，比正常模型好 60 倍——同时 **192 维里有 69 维已经死掉**。encoder 塌缩会让预测任务变得极其简单，这个指标在这里是**反向的**。成功率 34% 只是印证——它为什么低，第 13 页给的答案和我当时想的不一樣。

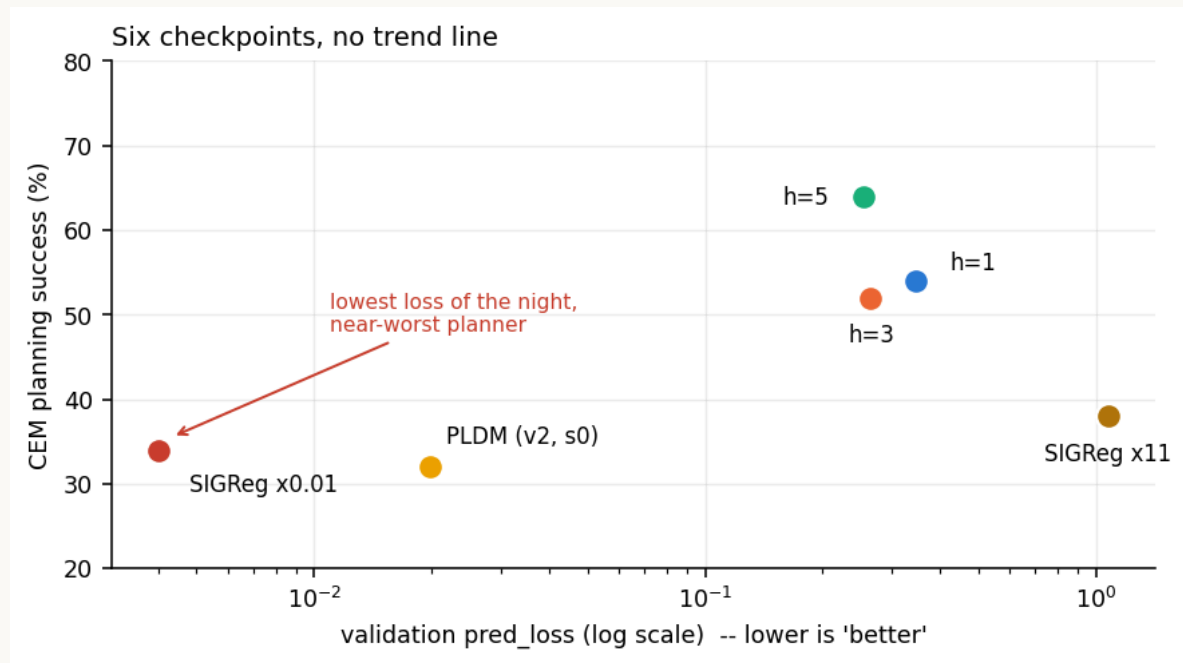
正则项没有失灵，它是被架空了

λ	pred_loss	sigreg_loss	emb 标准差	死维度	成功率
0.001	0.004	50.75	0.0012	69/192	34%
0.09 ★	0.266	5.16	0.325	0	52%
1.0	1.074	11.77	0.115	0	38%

SIGReg 一直在报警——它的值是 50.75，是健康状态的十倍。但权重只有 0.001，它对总 loss 的贡献只有 $0.001 \times 50.75 \approx 0.05$ ，而模型靠塌缩省下了 0.26 的预测误差。它出价出不过。

另一头是过度正则：不塌缩，但 embedding 被压得太紧（标准差 $0.325 \rightarrow 0.115$ ），区分状态的能力变弱。论文选的 0.09 正好落在这个倒 U 的顶点。记住成功率的这个形状：**34 / 52 / 38**。

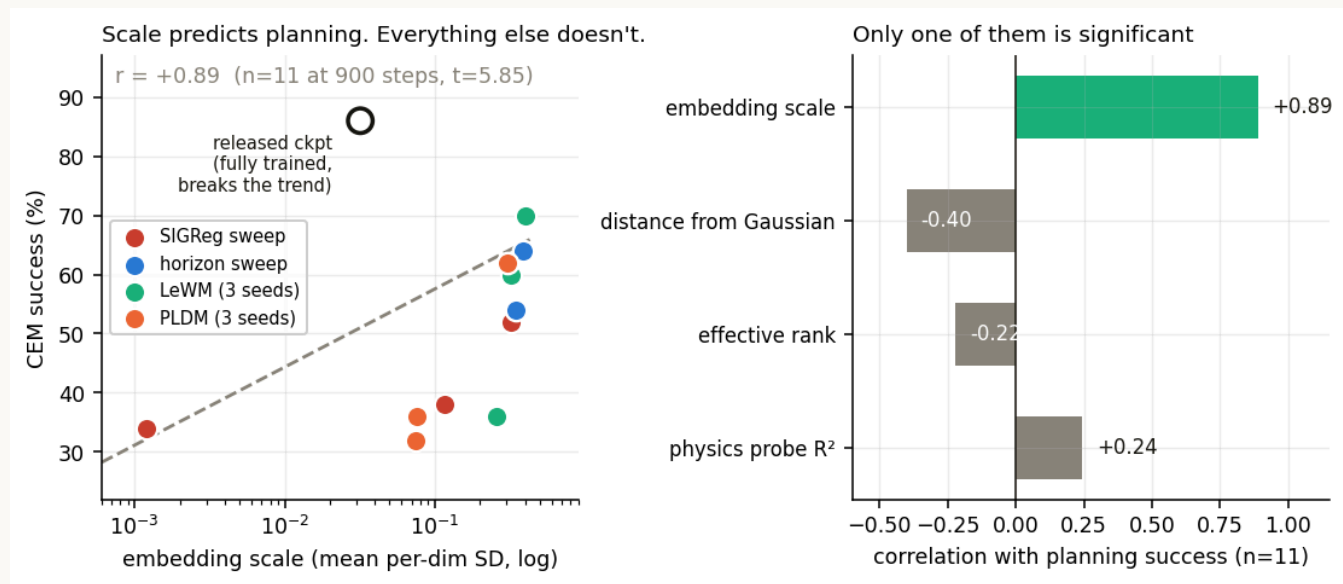
六个 checkpoint，没有趋势线



如果验证 loss 是可靠的代理指标，横跨三个数量级应该呈现一条向右下的趋势线。**它没有。**于是我手上只剩成功率这一个指标，而它的噪声是 ± 17 个百分点。与其再多跑 seed 硬扛噪声，不如换一个问题：**到底是什么决定规划成不成功？**

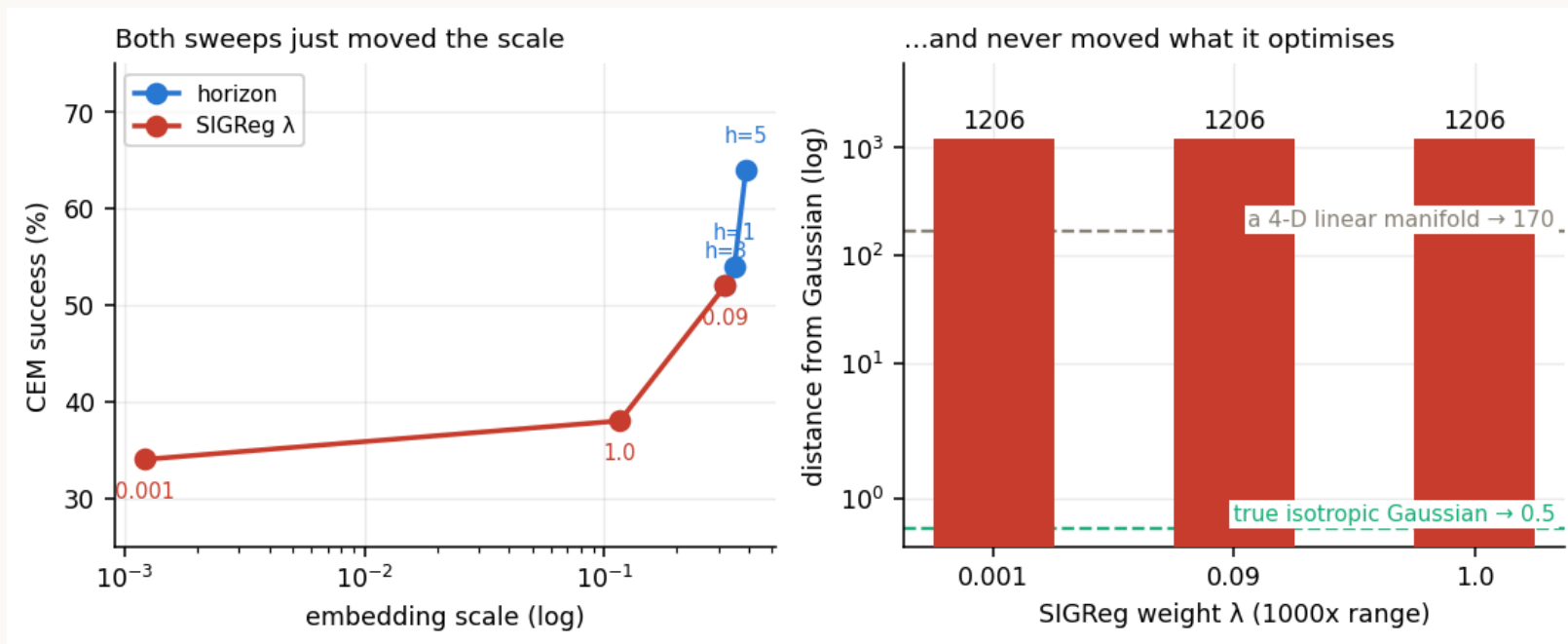
实验四 —— 换一批噪声小得多的指标

不训练任何新模型：在已有的 12 个 checkpoint 上、3000 帧留出数据上测四个量—— Epps-Pulley 正态性 (**SIGReg 声称要优化的东西**)、岭回归探针 embedding $\rightarrow$ 智能体位置的留出 R^2 (论文自己的 probing 口径)、有效秩、embedding 的绝对尺度。指标先在已知分布上校准过：真高斯给 0.5，4 维线性流形嵌入 192 维给 170.6。



表征里有多少物理信息，几乎不决定它能不能规划；表征有多大才决定。尺度 $r = +0.89$ ；探针 $R^2 +0.24$ 、有效秩 -0.22 、离高斯距离 -0.40 ，全部不显著。留一法：删掉任意一个 checkpoint，尺度那条的 r 始终落在 $[+0.87, +0.95]$ ，**不是离群点撑起来的**。这也推翻了我实验三给的解释：塌缩那个 checkpoint 探针 R^2 是 0.467，健康的是 0.497，**塌掉的不是信息**。

于是：前三组实验测的是同一个变量

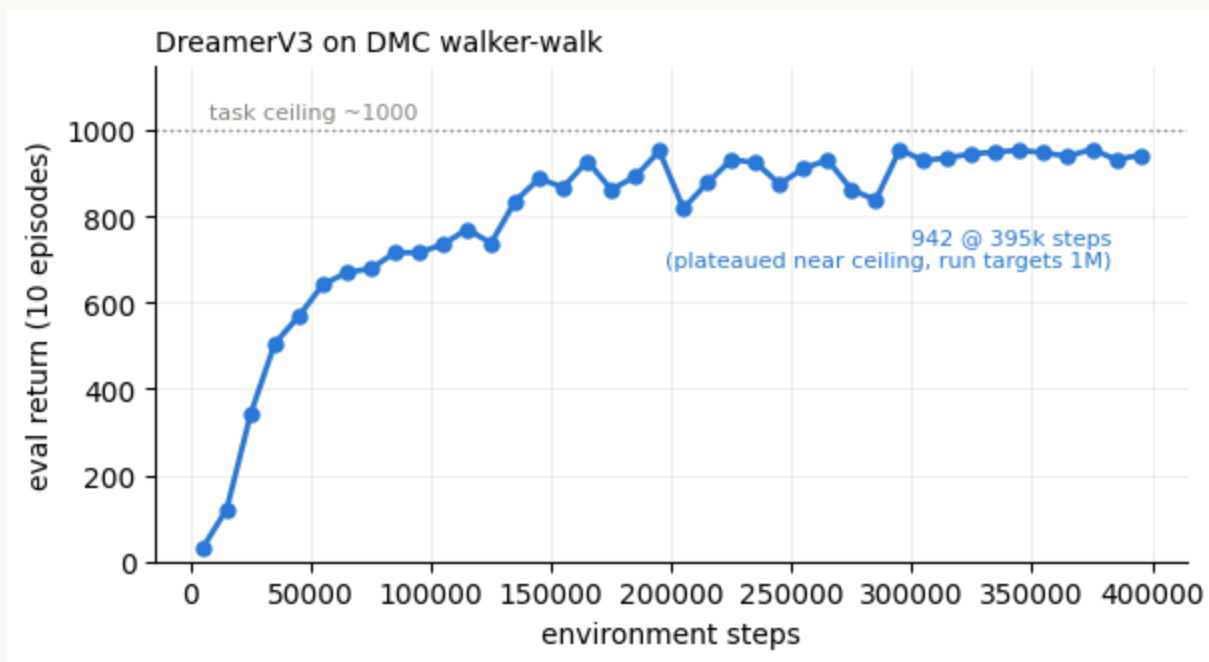


实验一的成功率排序（52 / 54 / 64）与它们的尺度排序（0.319 / 0.348 / 0.389）完全一致；实验三成功率的倒 U（34 / 52 / 38）与尺度的倒 U（0.0012 / 0.319 / 0.116）同形。三个不同的旋钮，落到同一个中介变量上。加 seed 救不了这些实验——45 个 seed 只会得到一个混杂量的精确估计。问题不在功效，在设计。

顺带的收获（右图）：整个 1000 倍权重扫描里，SIGReg 的高斯性统计量是 **1206 / 1206 / 1206**，纹丝不动，而真高斯是 0.5、4 维线性流形是 170。它实际维持的是 embedding 尺度，不是高斯化——论文命名的机制和实际生效的机制不是一回事。

规模验证 —— DreamerV3 在 DMC walker-walk

手写版证明我理解每个组件；这一条是官方实现在真实基准上的完整曲线。



959

855k 步时的评估回报（满分约 1000）

约 30 万步起进入平台期，稳定在满分（约 1000）附近。训练继续跑向 1M。

真正花掉时间的地方

把 **torch** 软链进别人的 **venv** 复用。uv 在处理一个没锁版本的 `torch>=2` 时顺着软链写穿了，悄悄把另一个项目的 torch 从 2.7 升到 2.14，而那个项目当时还有个跑了一天的进程在跑。

embed_dim 不是自由超参数。projector 的输入宽度取自它，而 ViT-tiny 的宽度固定在 192。我原计划的第一个消融直接崩在矩阵维度上——要正经扫它必须同步缩放骨干网络，那就不再是单变量消融了。

stdout 缓冲藏了两小时进度。终端日志卡在 45000 步一个多小时，而显式落盘的 `metrics.jsonl` 早就过了 80000 步。

完整记录在仓库的 engineering log 里。写下来是因为——出问题的地方，恰恰是没人愿意写下来的地方。

什么站得住，什么站不住

站得住（都是直接测量）

塌缩本身：embedding 标准差 0.0012 对 0.325，差 270 倍；死维度 69 对 0。没有噪声解释空间。

尺度与规划成功率的相关： $r = 0.89$ ， $t = 5.85$ ， $n = 11$ 。

SIGReg 的高斯性统计量在 1000 倍权重扫描下完全不动（1206 / 1206 / 1206）。

站不住

前三组的成功率结论**全部作废**——它们测的是同一个混杂变量（embedding 尺度）。

尺度是标记，不是原因。规划器对尺度严格不变（代价是 MSE、选精英是纯排序），所以它必然在代理别的东西——而那是什么，这个项目没回答。

我最初写下的那个“下一步”**本身是无效的**：评估时缩放 embedding 按构造必然得零。有效替代是评估期噪声注入，或训练期钉死尺度后重跑。

全部结论都在 900 步的欠训练模型上。官方 checkpoint 尺度只有 0.032 却拿 86%——这条规律在训练充分后就不成立。

没有 Dreamer 与 LeWorldModel 的正面对比：目标函数和评估指标都不共享。

下一步的优先级被改写了两次：先是从“扫更多超参数”改成“把评估规模做够”，再从那里改成“先搞清楚主指标到底被什么决定”。**回头看，第四组应该是第一组。**